Operational Governance & Versioning Document

Real-Time Cyber Anomaly Detection System

Version: 2.0 **Date:** 2025-12-12 **Authors:** Francesca Craievich, Lucas Jakin, Francesco Rumiz

---

1. Introduction

This document describes the operational governance practices adopted by the team throughout the project development. It covers version control strategies, CI/CD automation, model lifecycle management, monitoring practices, and deployment procedures.

---

2. Version Control Strategy

2.1 System and Platform

Aspect	Details
Version Control System	Git
Hosting Platform	GitHub
Repository	https://github.com/francescacraievich/real-time-cyber-anomaly-detection

2.2 Branching Strategy

The team follows a **feature branch workflow** with protected main branch.

Branch Types

Branch Type	Pattern	Purpose	Example
Main	`main`	Production-ready code, protected	`main`
Feature	`feature/<issue>-<desc>`	New features	`feature/5-anomaly-detection-model`
Fix	`fix/<issue>-<desc>`	Bug fixes	`fix/7-fix-formatter-classes`
Docs	`docs/<issue>-<desc>`	Documentation	`docs/8-update-documentation`
Test	`test/<issue>-<desc>`	Test additions	`test/6-add-tests-for-initializer`

Active Branches

Branch	Purpose
`feature/5-anomaly-detection-model-implementation`	OCSVM model development and training
`feature/4-add-new-features-for-data-engineering`	Feature engineering functions
`feature/streamlit-flask-integration`	Dashboard and API development
`feature/fix-module-imports`	Fix import issues and gzip support
`docs/8-update-documentation`	Documentation updates (SSD, proposal, governance)
`fix/7-fix-formatter-classes`	Bug fixes for formatter classes
`test/6-add-tests-for-initializer-and-formatter-classes`	Unit tests for data processing

2.3 Git Workflow

Complete Workflow:

1. **Issue Creation**: Each task is documented as a GitHub Issue with clear acceptance criteria
2. **Branch Creation**: Developer creates a branch following the naming convention, linking to the issue number
3. **Development**: Commits are made with descriptive messages following conventional commit format
4. **Pull Request**: PR is opened with description of changes, linking to the issue
5. **Code Review**: At least one team member reviews the code, providing feedback
6. **CI Validation**: GitHub Actions runs linting, tests, and documentation build
7. **Merge**: After approval and green CI, the branch is merged to main
8. **Cleanup**: Feature branch is deleted after successful merge

2.4 Commit Message Conventions

Prefix	Usage	Example
`feat:`	New features	`feat: add drift detection module`
`fix:`	Bug fixes	`fix: handle missing values in parser`
`docs:`	Documentation	`docs: update SSD with architecture diagram`
`test:`	Test additions	`test: add unit tests for rate features`
`refactor:`	Code refactoring	`refactor: simplify feature extraction pipeline`
`style:`	Formatting	`style: apply black formatting`
`chore:`	Maintenance	`chore: update dependencies`

2.5 Tagging and Releases

Tag Pattern	Purpose	Example
`v<major>.<minor>.<patch>`	Semantic versioning for releases	`v1.0.0`
`model-v<n>`	Model version tracking	`model-v4`

3. CI/CD & Automation

3.1 Platform

CI/CD Platform: GitHub Actions

3.2 Workflows

CI Workflow (`.github/workflows/ci.yml`):

Trigger	Jobs
Push to all branches	`docs`, `test`
Pull requests to `main`	`docs`, `test`

3.3 CI Jobs Details

Documentation Job (`docs`):

```
- Checkout code
- Setup Python 3.11
- Install dependencies
- Run mkdocs build --strict
```

Test Job (`test`):

```
- Matrix strategy: Python 3.11, 3.12
- Install dependencies from requirements.txt and requirements-dev.txt
- Linting: black, isort, flake8 checks on src/ and tests/
- Testing: pytest with coverage reporting
- Coverage: Upload to Codecov
```

3.4 Code Quality Tools

Tool	Purpose	Configuration
Black	Code formatter	`pyproject.toml` (line-length: 88)
isort	Import sorter	`pyproject.toml` (profile: black)
flake8	Linter	`.flake8` (max-line-length: 88)

Tool	Purpose	Configuration
Codecov	Coverage reporting	`.codecov.yml` (patch target: 40%)

3.5 Code Coverage Configuration

Codecov Configuration (`.codecov.yml`):

```
coverage:
  status:
    patch:
      default:
        target: 40%  # Lowered threshold for patch coverage
```

Rationale: The 40% patch coverage target was selected to enable rapid iteration during early development phases while maintaining a baseline quality standard. As the project matures, this threshold may be increased.

3.6 CI/CD Security

The CI workflow implements security best practices:

```
permissions:
  contents: read  # Minimal permissions
```

This restricts the workflow to read-only access, preventing malicious code from modifying the repository.

---

4. Model Lifecycle Governance

4.1 Model Versioning

Model artifacts are stored in `src/model/` directory with pickle serialization:

File	Description	Contents
`oneclass_svm_model.pkl`	Trained OCSVM model	Scikit-learn OneClassSVM object
`oneclass_svm_preprocessor.pkl`	Preprocessing pipeline	ColumnTransformer (RobustScaler + OneHotEncoder)
`oneclass_svm_config.pkl`	Model configuration	Threshold, feature list, hyperparameters

4.2 Model Registry Approach

The project uses a file-based model registry with the following structure:

```
src/model/
├── oneclass_svm_model.pkl        # Current production model
├── oneclass_svm_preprocessor.pkl # Associated preprocessor
├── oneclass_svm_config.pkl       # Configuration and metadata
└── main.py                       # Training entry point
```

Model Loading Strategy:

- `fit_or_load()` method checks for existing model files
- If files exist and are valid, model is loaded from disk
- If files don't exist or are corrupted, model is retrained
- Model artifacts are version-controlled in Git (tracked in repository)

4.3 Reproducibility

Aspect	Implementation
Random State	`random_state=42` for all random operations
Train/Val Split	80/20 split with fixed seed
Dependencies	Pinned versions in `requirements.txt`
Data Versioning	Processed datasets stored in `data/processed/`
Configuration	Hyperparameters stored in `oneclass_svm_config.pkl`

4.4 Experiment Tracking

Current Approach:

- Model performance logged to console during training
- Evaluation metrics computed via `evaluate_model_performance()`
- Results documented in commit messages and PR descriptions

Future Enhancement:

- Integration with Neptune.ai for comprehensive experiment tracking
- Automatic logging of hyperparameters, metrics, and artifacts
- Comparison dashboards for model versions

4.5 Model Retraining Policy

Trigger	Condition	Action
Scheduled	Monthly	Retrain with latest data
Drift Detected	ADWIN triggers drift alert	Evaluate and retrain if performance drops
Performance Drop	Accuracy < 80%	Immediate retraining
Data Update	New labeled samples available	Retrain with expanded dataset

5. Monitoring & Maintenance Plan

5.1 Monitoring Stack

Component	Purpose	Location
Prometheus	Metrics collection and storage	Docker container (`monitoring/`)
Grafana	Visualization dashboards	Docker container (`monitoring/`)
prometheus-client	Python metrics library	`src/monitoring/metrics.py`

5.2 Docker Setup

```
# Start monitoring stack
cd monitoring/
docker-compose up -d

# Access points:
# - Prometheus: http://localhost:9090 (metrics storage and querying)
# - Grafana: http://localhost:3000 (visualization dashboards)
```

5.3 Prometheus Metrics

The following custom metrics are exposed by the application at `/metrics` endpoint:

Metric	Type	Description	Labels
`anomaly_detection_predictions_total`	Counter	Total predictions	`severity` (GREEN/ORANGE/RED)
`anomaly_detection_prediction_latency`	Histogram	Inference latency (seconds)	-
`anomaly_detection_f1_score`	Gauge	Current F1 performance score	-
`anomaly_detection_drift_status`	Gauge	Drift status (0=stable, 1=drift)	-
`anomaly_detection_anomaly_rate`	Gauge	Rolling anomaly rate percentage	-

5.4 Drift Detection

Algorithm: ADWIN (Adaptive Windowing) from River library

Configuration:

Parameter	Value	Description

Parameter	Value	Description
`delta`	0.002	Confidence parameter (lower = more sensitive)
`window_size`	100	Sliding window size for anomaly rate
`change_threshold`	0.08	Minimum 8% change to trigger alert

Detection Methods:

1. **ADWIN Statistical Test**: Monitors anomaly rate stream for distribution changes
2. **Threshold-based**: Triggers when rolling anomaly rate changes by >8%

Drift Status:

Status	Condition	Response
STABLE	No significant change	Normal operation
UNSTABLE	Drift detected	Alert in ML Dashboard, recommend retraining

5.5 Incident Response

Incident	Detection	Response	Owner
Model drift detected	ADWIN alert	Evaluate performance, retrain if needed	ML Engineer
High false positive rate	User feedback, metrics	Adjust threshold, retrain model	ML Engineer
API unresponsive	Health check failure	Restart service, check logs	Software Developer
Data quality issues	Validation pipeline	Investigate source, fix pipeline	Data Scientist

5.6 Maintenance Schedule

Task	Frequency	Owner
Review drift detection alerts	Daily	ML Engineer
Check Prometheus/Grafana health	Weekly	Software Developer
Review model performance metrics	Weekly	ML Engineer
Update dependencies	Monthly	All Team
Full model retraining	Monthly or on drift	ML Engineer
Security updates	As needed	All Team

6. Deployment Strategy

6.1 Streamlit Cloud Deployment

The application dashboards are deployed on Streamlit Cloud for public access:

Dashboard	URL	Purpose	Target Users
Anomaly Dashboard	https://dashboard-anomalydetection.streamlit.app/	Real-time threat visualization	Security Analysts, SOC Team
ML Monitoring	https://monitoring-model.streamlit.app/	Model health monitoring	ML Engineers, Data Scientists

6.2 Local Development Setup

```
# Clone repository
git clone https://github.com/francescacraievich/real-time-cyber-anomaly-
detection.git
cd real-time-cyber-anomaly-detection

# Install dependencies
pip install -r requirements.txt
pip install -r requirements-dev.txt  # For development

# Run Flask API
python -m src.dashboard.flask_api

# Run Streamlit dashboard (in separate terminal)
streamlit run src/dashboard/streamlit_app.py

# Run monitoring dashboard
streamlit run src/dashboard/streamlit_monitoring.py
```

6.3 Docker Monitoring Stack

```
# Start Prometheus and Grafana
cd monitoring/
docker-compose up -d

# Stop monitoring stack
docker-compose down
```

6.4 Documentation Deployment

Platform	URL	Trigger
GitHub Pages	https://francescacraievich.github.io/real-time-cyber-anomaly-detection/	Push to `main` branch

Build Command: `mkdocs build --strict`

---

7. Security

7.1 Security Policy

The project includes a SECURITY.md file that defines:

- How to report vulnerabilities responsibly
- Security measures implemented
- Best practices for deployment

7.2 GitHub Security Features

Feature	Status	Description
Code Scanning (CodeQL)	Enabled	Automatic vulnerability detection
Secret Scanning	Enabled	Detects committed secrets
Security Advisories	Enabled	Vulnerability disclosure
Dependabot Alerts	Available	Dependency vulnerability alerts

7.3 Application Security Measures

Measure	Implementation
No Debug Mode	Flask runs with `debug=False`
No Stack Traces	Exceptions return generic messages
Input Validation	API endpoints validate parameters
CORS Configuration	Controlled cross-origin access
Minimal CI Permissions	`contents: read` only

---

8. Testing Strategy

8.1 Test Structure

Test Suite	Location	Coverage
`test_precalculations/`	`tests/`	Feature calculation functions
`test_aggregations/`	`tests/`	Aggregation metrics
`test_formatters/`	`tests/`	Data formatters
`test_model/`	`tests/`	ML model and drift detection
`test_dashboard/`	`tests/`	Flask API tests

Total Tests: 116

8.2 Running Tests

```
# Run all tests
pytest tests/

# Run with coverage
pytest tests/ --cov=src --cov-report=term

# Run specific test suite
pytest tests/test_model/

# Run with verbose output
pytest tests/ -v
```

8.3 Test Automation

Tests are automatically executed on:

- Every push to any branch
- Every pull request to `main`

Test failures block PR merges to `main` branch.

---

9. Pull Requests & Code Review

9.1 PR Process

Step	Action	Responsible
1	Create PR with description	Developer
2	Link to related issue	Developer
3	CI checks run automatically	GitHub Actions
4	Request review from team member	Developer
5	Address review feedback	Developer
6	Approve PR	Reviewer
7	Merge to main	Developer or Reviewer
8	Delete feature branch	Developer

9.2 PR Examples

PR	Branch	Description

PR	Branch	Description
#15	`feature/fix-module-imports`	Fix module import errors and add gzip support
#13	`feature/4-add-new-features-for-data-engineering`	Feature engineering functions

9.3 Issue Tracking

- Issues are linked to branches via naming convention (e.g., `feature/5-...` links to issue #5)
- Issues track features, bugs, documentation, and tests
- Branches are deleted after merge

Document Version History

Version	Date	Changes
1.0	2025-11-29	Initial governance document
1.1	2025-12-11	Added CI/CD details, monitoring stack, testing section
1.2	2025-12-11	Added security section
1.3	2025-12-12	Added Prometheus Metrics, Drift Detection, Agile Methodology
2.0	2025-12-12	Major reorganization: focused on operational governance only, added Model Lifecycle Governance (§4), consolidated Monitoring & Maintenance (§5), moved Agile/CRISP-DM to Project Proposal, moved technical architecture to SSD